

REPORT HOMEWORK CMRM2024

Francesco Crociani and Luca Trapella

November 2024

Contents

1	Introduction	5
2	Dataset	7
3	Data processing	11
3.1	Feature Vector	12
4	Support Vector Machine (SVM)	17
5	Improving the dataset	23
5.1	Dataset Balancing	23
5.2	Data Augmentation	23
5.3	Additional possible Augmentations	27
6	Tuning of hyperparameters and final results	29
6.1	Accuracy	29
6.2	Confusion matrix and classification report	29
6.3	Tuning Hyperparameters	31
6.4	Improvement possibilities	35

Chapter 1

Introduction

The focus of this report is the classification of electrocardiogram (ECG) signals based on their rhythmic characteristics. The objective is to develop an algorithm capable of categorizing ECG waveforms into five types of arrhythmia, relying solely on rhythmic features. To achieve this, we will extract a detailed feature vector and train a classifier using the MIT-BIH dataset. Specifically, the task will involve training a Support Vector Machine (SVM), evaluating its performance, refining the model to improve results, and drawing conclusions from the analysis.

Chapter 2

Dataset

When working with machine learning or deep learning algorithms, the initial step involves loading, analyzing, and preprocessing the dataset. These models are usually developed using Python scripts, with datasets stored locally. In many cases, datasets are provided in .csv format, sparing us from the need to download large volumes of .wav files. For our ECG classification task, we will use the MIT-BIH dataset, which is available in the "dataset" folder. The dataset is designed for ECG signal classification, specifically to identify and classify ECG waveforms into five distinct arrhythmia classes. This is crucial for detecting and diagnosing heart rhythm disorders. Typically, the 80% of the dataset is used for training, while the 20% for testing the model. In our case, data are already divided into two subsets as we have two distinct files.

Structure of the Dataset

The provided dataset contains ECG signal data, where each row corresponds to a single sample. The structure of the data is as follows:

- The first 187 columns of each row contain the values of the ECG signal, representing the waveform over time.
- The last column (index 188) contains the class index associated with the signal.

The dataset includes five distinct classes, which represent different types of ECG signals: Normal (N), Fusion of paced and normal (S), Premature ventricular contraction (V), Atrial premature (F) and Fusion of ventricular and normal (Q). Below is a plot of an histogram representing the number of signals for each class in the train and test dataset.

Each ECG signal is sampled at a frequency of 360 Hz, providing a high-resolution representation of the waveform. Below is an example plot of one signal, illustrating how the values in the first 187 columns form the ECG waveform. We will not use the whole dataset for training our classifier because this can easily occupy the whole RAM at disposal to Jupyter Notebook. Due to this reason, the performance of our classifier will probably not be the best possible but still

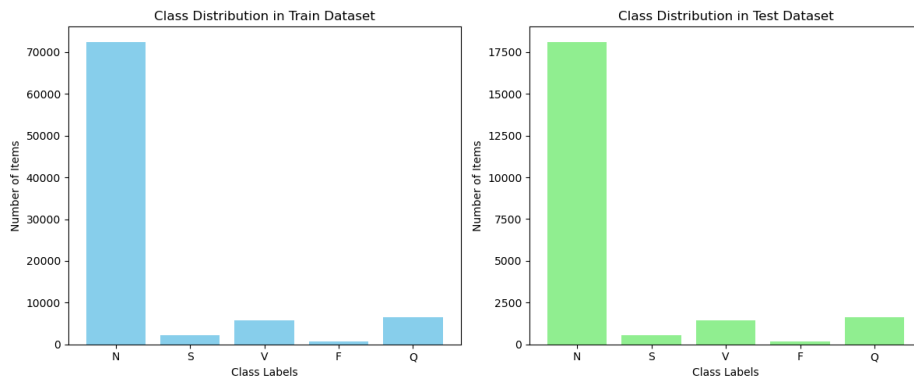


Figure 2.1: Class distribution of train and test dataset

Train Dataset

- **Data Shape:** (87553, 188)
- **Total Items:** 16.459.964 (87553 rows × 188 columns)
- **Number of Classes:** 5
- **N:** 72470, **S:** 6431, **V:** 5788, **F:** 2223, **Q:** 641

Test Dataset

- **Data Shape:** (21891, 188)
- **Total Items:** 4.115.508 (21891 rows × 188 columns)
- **Number of Classes:** 5
- **N:** 18117, **S:** 1608, **V:** 1448, **F:** 556, **Q:** 162

Figure 2.2: Train and Test set shape

sufficient for the scope of this assignment. So we subsample them creating new dataframes.

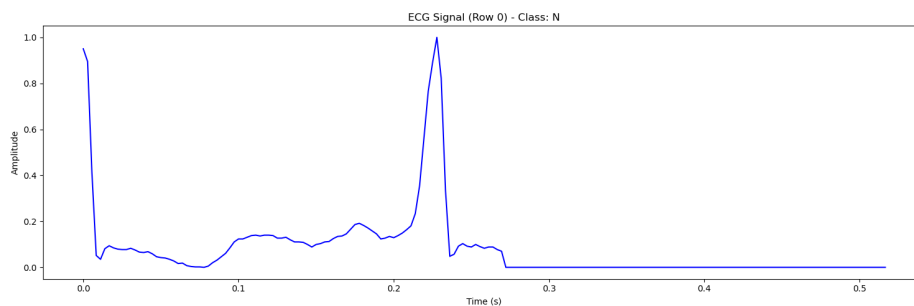


Figure 2.3: First ECG signal of the train dataset

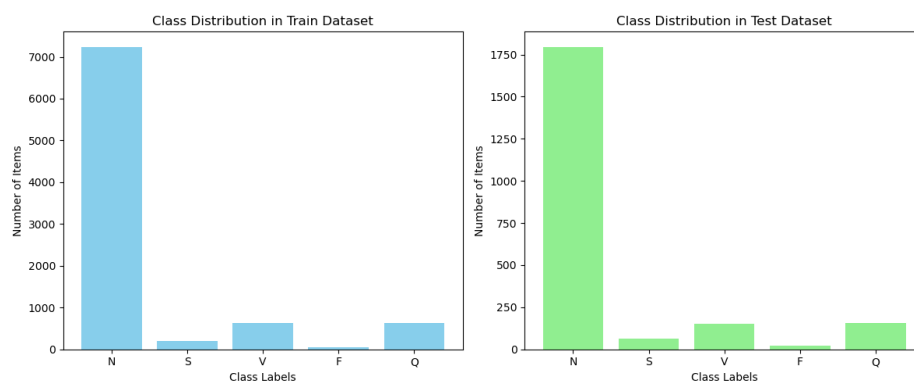


Figure 2.4: Class distribution of new train and test dataset

New Train Dataset

- **Data Shape:** (8755, 188)
- **Total Items:** 1.645.940
- **Number of Classes:** 5
- N: 7240, S: 635, V: 622, F: 201, Q: 57

New Test Dataset

- **Data Shape:** (2189, 188)
- **Total Items:** 411.532
- **Number of Classes:** 5
- N: 1796, S: 157, V: 153, F: 62, Q: 21

Figure 2.5: Train and Test set new shape

Chapter 3

Data processing

Now that we have loaded the dataset, we are able to process it. In particular, different curves will be characterized by different dynamic ranges. To ensure consistent representation of the ECG signals and improve the generalization capability of the classification model, normalization is applied to the data. This step aligns the dynamic ranges of the signals and ensures that the waveforms cross the zero-axis multiple times and have the same codomain. Below is the code implementing what we have discussed.

```
1 from sklearn.preprocessing import MinMaxScaler
2 train_list = train_df.values.tolist()
3 test_list = test_df.values.tolist()
4 # Normalizing data, define the MinMaxScaler with feature
   range (-1, 1)
5 scaler = MinMaxScaler(feature_range=(-1, 1))
6 # Fit the scaler on the train list and transform it, fit
   finds the
7 # parameters related to the train set, and transform data
8 # rescaling it using the previously found parameters
9 train_set = scaler.fit_transform(train_list)
10 # Transform the test list using the same scaler, we use
    only
11 # transform because it uses existing parameters found in
    the
12 # trainset in order to prevent data leakage
13 test_set = scaler.transform(test_list)
```

For the training data, the **fit_transform** method was applied to **train_list**. This method calculates the scaling parameters (minimum and maximum values) and transforms the data based on these parameters. This step ensures the model learns the appropriate normalization bounds from the training set. For the test data, the **transform** was applied to **test_list**. This method normalizes the test data using the scaling parameters derived from the training data. This approach avoids introducing bias or information leakage from the test set into the training process.

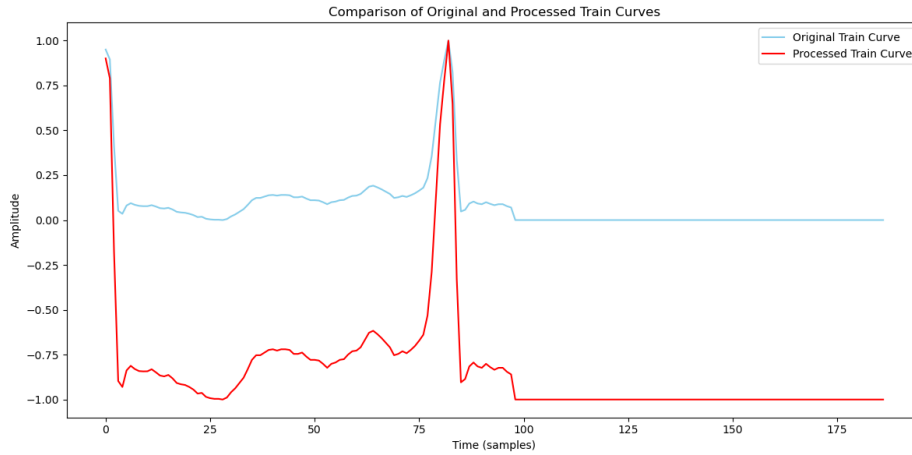


Figure 3.1: Comparison of the first original and processed train curve

3.1 Feature Vector

To accomplish classification based on ECG waveforms and rhythmic features, we will define a function `compute_feature_vector` that extracts multiple features such as standard deviation, mean, zero-crossing rate, spectral flux, and others. Below is an explanation of how to implement this function and the steps involved:

1. Compute the **standard deviation** and **mean** of the ECG waveform x to understand the variability and average amplitude of the signal.
2. **Zero-Crossing Rate (ZCR)** is defined as the rate at which the signal changes its sign (crosses zero) within a frame. It is often used in signal processing to detect the frequency content or rhythmic properties of a signal. From a rhythmic perspective, ZCR can indicate how quickly the waveform oscillates, which can be crucial for classifying rhythmic patterns in ECG signals.
3. After computing the zero-crossing rate, we compute its standard deviation and mean, which gives us an indication of how variable and centered the zero-crossing rate is.
4. Compute the **STFT** of the waveform which transforms the signal into the frequency domain. The parameters `n_fft` and `hop_length` are used to specify the **window length** and **hop size**. In our case we will use `n_fft = N = 64` (a power of 2 smaller than 187) and `hop_length = H = N/4 = 16`. The result will be the magnitude (absolute value) of the STFT, which reflects how the signal's frequency content changes over time.
5. Compute the **Spectral flux**. This measures how much the frequency content of a signal changes over time, reflecting the "novelty" or "onset" of new frequency components. Without `librosa`, you can compute spectral flux as the sum of squared differences between consecutive frames of the

STFT magnitude, which is essentially a measure of the change in spectral content between frames. A high spectral flux indicates rapid changes in the frequency content, often associated with transient events like onsets in music. Low spectral flux indicates a steady spectrum, often associated with sustained sounds. For a spectrogram S (magnitude of the Short-Time Fourier Transform), the spectral flux at time frame (t) is given by:

$$SF(t) = \sqrt{\sum_{k=1}^N (|S(k, t)| - |S(k, t-1)|)^2} \quad (3.1)$$

where: $S(k, t)$ is the magnitude of the spectrum at frequency bin k and time t . N is the number of frequency bins.

6. Compute the standard deviation and mean of the spectral flux to quantify how much the frequency content varies over time.
7. Concatenate all the computed features (mean and standard deviation of ECG, zero-crossing rate, and spectral flux) into a single feature vector.

The implementation of the feature vector is shown in the following code using librosa for the spectral flux:

```

1 from librosa import amplitude_to_db
2 def compute_feature_vector(x, Fs, N, H):
3
4     # 1. Compute the standard deviation and the mean of
5     #     the ECG waveform
6     std_ecg = np.std(x)
7     mean_ecg = np.mean(x)
8
9     # 2. Compute the zero-crossing rate using librosa
10    zcr =
11        librosa.feature.zero_crossing_rate(x, frame_length=N,
12        hop_length=H) # Zero-crossing rate (1 x time), zcr
13        is an array of arrays of 12 elements each in our
14        case
15
16    # 3. Compute the standard deviation and mean of the
17    #     zero-crossing rate
18    std_zcr = np.std(zcr)
19    mean_zcr = np.mean(zcr)
20
21    # 4. Compute the STFT of the waveform using librosa
22    C = np.abs(librosa.stft(x, n_fft=N, hop_length=H)) #
23        Short-time Fourier transform
24    C_db = amplitude_to_db(C, ref=np.max) # Convert to dB

```

```

20     # 5. Compute the spectral flux using librosa (Onset
        strength)
21     spectral_flux = librosa.onset.onset_strength(S=C_db)
22
23     # 6. Compute the standard deviation and mean of the
        spectral flux
24     std_sf = np.std(spectral_flux)
25     mean_sf = np.mean(spectral_flux)
26
27     # 7. Define the feature vector by concatenating all
        the features
28     f_vector = np.array([mean_ecg, std_ecg, mean_zcr,
        std_zcr, mean_sf, std_sf])
29     f_vector = np.concatenate((f_vector, zcr[0]))
30     f_vector = np.concatenate((f_vector, spectral_flux))
31
32     return f_vector

```

Implementation of the spectral Flux without librosa

In order to implement it we can follow these steps:

1. Compute the Short-Time Fourier Transform of the signal to get a spectrogram
2. Take the absolute value of the STFT result to obtain the magnitude spectrogram.
3. Convert magnitudes to decibels or normalize them.
4. Compute the difference between consecutive frames.
5. **Half-Wave Rectification:** Set negative differences to zero.
6. **Aggregate Energy:** Sum the squared differences across frequency bins and take the square root.

Below is the implementation of spectral flux without librosa:

```

1  # Function to compute spectral flux
2  def compute_spectral_flux(stft_magnitude):
3  # Ensure positive differences only (half-wave
        rectification)
4  flux = np.maximum(0, np.diff(stft_magnitude, axis=1))
5  # Sum squared differences across frequency bins
6  flux = np.sqrt(np.sum(flux**2, axis=0))
7  return flux
8  # Generate the signal
9  Fs = 22050

```

```

10 t = np.linspace(0, 1, Fs)
11 signal = np.sin(2 * np.pi * 220 * t) + np.sin(2 * np.pi *
    440 * t)
12 # Short-Time Fourier Transform
13 n_fft = 2048
14 hop_length = 512
15 stft = np.abs(np.fft.rfft(signal, n=n_fft, axis=0))
16 # Compute spectral flux manually
17 spectral_flux_manual = compute_spectral_flux(stft)

```

Test feature vector on the train set and on the test set

Now that we have defined the function for extracting the feature vector, we can test it on the first waveform of the training set.

```

1 # Test the function with the first waveform of the
    training set
2 x = train_set[0] # First waveform of the train set
3 Fs = 360 # Sample rate (assumed, adjust according to your
    dataset)
4 N = 64 # Window length for STFT
5 H = 16 # Hop size for STFT, H=N/4
6
7 # Compute the feature vector for the first waveform
8 f_vector = compute_feature_vector(x,Fs,N,H)
9 print(f_vector) # 30 elements, zcr and spectral flux have
    12 elements each
10
11
12 # Plot the feature vector
13 plt.figure(figsize=(10, 6))
14 plt.bar(range(len(f_vector)), f_vector, color='orange')
15 plt.xlabel('Feature Index')
16 plt.ylabel('Feature Value')
17 plt.title('Feature Vector of the First ECG Signal')
18 plt.show()

```

Below is the graph representing the feature vector for the training set.

If we apply it for all the waveforms in train set and test set we will obtain these shapes: shape of train feature vectors = (8755, 30); shape of test feature vectors = (2189, 30)

Comparison with the novelty function

A novelty function is designed to detect changes or transitions in a signal over time. These changes typically correspond to perceptually significant events, such as onsets of musical notes, beats, or changes in rhythm or timbre. The goal is to produce a representation that highlights these transitions, enabling further

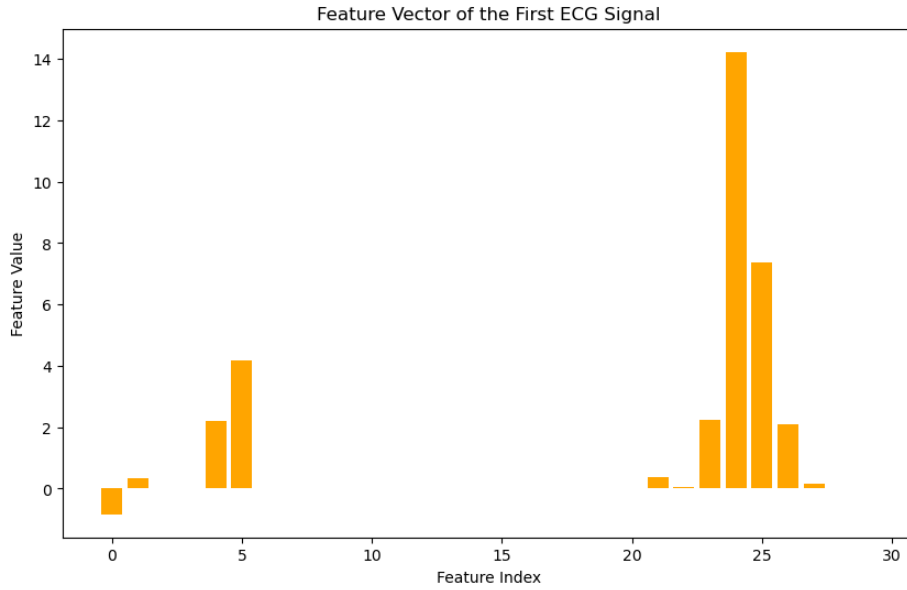


Figure 3.2: Feature vector of the first waveform of train set.

tasks like segmentation, beat tracking, or rhythm analysis. Adding different rhythmic features enriches the feature set and allows the model to capture various dimensions of the signal’s dynamics and structure, like in our feature vector. For example:

- The Zero Crossing Rate (ZCR) can characterize the rapidity or noisiness of the signal transitions.
- The Spectral Flux measures changes in the spectral content, revealing how the energy of the signal evolves across time and frequency.
- Mean and standard deviation of these features provide global summaries of rhythmic behavior.

These rhythmic features can reveal complementary information that is not directly visible in the time-domain signal or basic statistical features, improving the classifier’s ability to generalize across diverse waveforms. The trade-offs of extracting rhythmic features are:

1. **Computational Complexity:** Extracting additional rhythmic features (e.g., spectral flux or STFT) can increase the computational cost, particularly for large datasets or real-time applications.
2. **Risk of Overfitting:** Introducing too many features may lead to overfitting, especially if some features are redundant or irrelevant to the classification task.
3. **Feature Selection:** Not all rhythmic features may contribute equally to classification performance. Adding irrelevant or noisy features can degrade model performance or require additional steps like feature selection.

Chapter 4

Support Vector Machine (SVM)

After extracting the vector containing different rhythmic features, we are going to implement a **Support Vector Machine** model in order to accomplish classification. SVMs are commonly used within classification problems. They distinguish between two classes by finding the optimal hyperplane that maximizes the margin between the closest data points of opposite classes. Like all machine learning models, SVMs are characterized by **hyperparameters**: settings that control the behavior of the algorithm and significantly influence its performance. These hyperparameters are not learned from the data but must be specified before training and then be tuned during the training process. In our case we are focusing on the **regularization parameter** and the **kernel type**. The regularization parameter describes how the model penalizes classification errors, while the kernels are functions used to transform the data higher-dimensional space to enable linear separation. The choice of kernel function, such as linear kernels, polynomial kernels, radial basis function (RBF) kernels, or sigmoid kernels, depends on data characteristics and the specific use case. Each kernel has a unique behavior and is suited for specific distributions or characteristics of the dataset:

- **Linear Kernel**: uses a linear function to separate the data, is computationally efficient with large datasets but is not suitable for complex problems where class separation is non-linear.
- **Polynomial Kernel**: applies a polynomial transformation to the data so it's suitable for moderately complex problems but the computational complexity grows exponentially with higher degrees.
- **Radial Basis Function (RBF) Kernel**: uses a function based on the squared euclidian distance and a parameter γ , which describes the influence of each training sample to obtain data separation. It's very flexible for complex problems and suitable for various types of data distributions, on the other hand requires tuning of the γ parameter, which can be computationally expensive.

$$K(x, x') = e^{\left(-\frac{\|x-x'\|^2}{2\sigma^2}\right)} \quad (4.1)$$

where γ is defined as $-\frac{1}{2\sigma^2}$.

- **Sigmoid Kernel:** uses the sigmoid function to separate the data similarly to the activation functions used in neural networks. Usually less popular and often less effective compared to RBF and the parameters α and β require careful tuning.

$$K(x, x') = \tanh(\alpha \cdot xx' + \beta) \quad (4.2)$$

In our case we initially implemented a SVM model with regularization parameter $c = 10$ and kernel type **rbf** with the default value of γ . Later on we will tune the hyperparameters trying different model settings in order to obtain the most efficient combination. The SVM model is trained on the feature vector and a training accuracy of 89.01% is obtained with the following code.

```

1 from sklearn.svm import SVC
2 from sklearn.metrics import accuracy_score
3 import joblib
4
5
6 C = 10
7 kernel = 'rbf'
8
9
10 # Train SVM
11 svc = SVC(C=C, kernel=kernel)
12 svc.fit(train_fvector, labels_train)
13
14 # Save the model. In this way, I can load it without the
    need for training from scratch.
15 model = f'model/svc_{kernel}C{C}N{N}H{H}.joblib'
16 joblib.dump(svc, model)
17 print(f"Model saved to {model}")
18
19 # Load the model
20 svc_loaded = joblib.load(model)
21
22 # Compute accuracy on the training set
23 train_predictions = svc_loaded.predict(train_fvector)
24 train_accuracy = accuracy_score(labels_train,
    train_predictions)

```

The training process could easily encounter problems such as overfitting which represents the weakness of the model when it has to generalize from training data to unseen test data. Overfitting occurs when the model provides very accurate prevision on the training data but can't do the same for test data and implies that the model has acquired irrelevant patterns that are useless for generalizing.

After the first training process we test the SVM classifier on the test set obtaining 87.8% accuracy, which alone seems like a great value.

```

1     # Test the classifier
2     test_predictions = svc.predict(test_fvector)
3
4     # Calculate accuracy
5     test_accuracy = accuracy_score(labels_test,
6                                     test_predictions)
7     # Print the accuracy
8     print(f"Test Accuracy: {test_accuracy*100:.2f}%")

```

However the mere accuracy is not enough to verify the performance of a classifier and this can be seen from the plot of the **Confusion Matrix**, a table used to evaluate the performance of a classification model. It summarizes the predictions made by the model compared to the actual labels, giving a detailed breakdown of correct and incorrect classifications. For multiclass classification, the confusion matrix extends to an $n \times n$ table, where n is the number of classes.

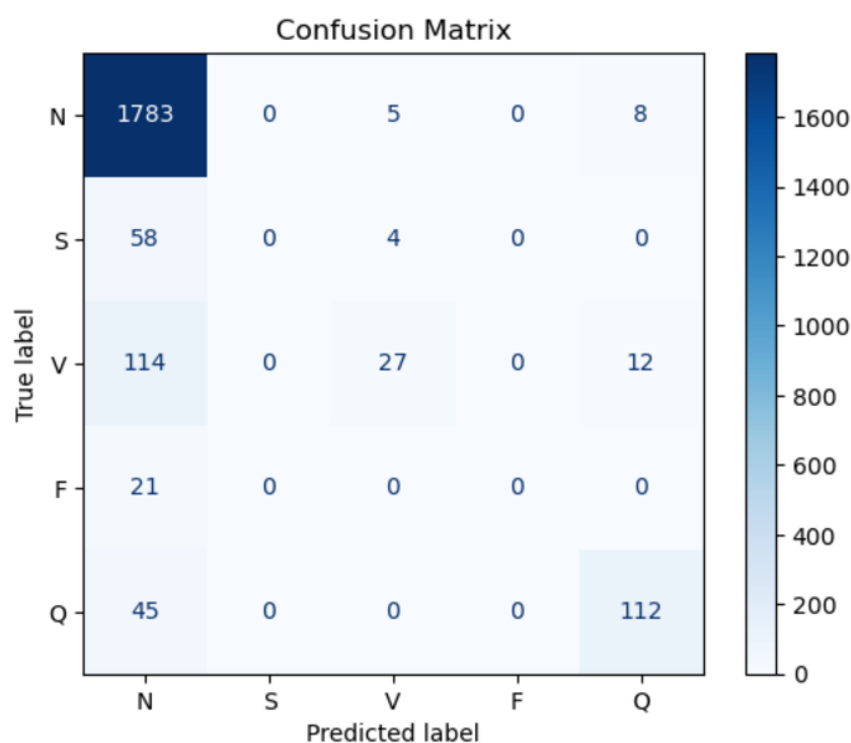


Figure 4.1: Confusion Matrix

```

1     from sklearn.metrics import ConfusionMatrixDisplay
2
3     # Calcola e visualizza la matrice di confusione
4     ConfusionMatrixDisplay.from_estimator(
5         estimator=svc_loaded,          # Modello caricato
6         X=test_fvector,                # Feature del set di test

```

```

7     y=labels_test,                # Etichette reali del set
      di test
8     display_labels=label_names.values(), # Nomi delle
      classi
9     cmap='Blues',                # Colormap per la matrice
10    colorbar=True                # Barra dei colori
11 )
12
13 plt.title("Confusion Matrix")
14 plt.show()

```

The class N has a much bigger presence in the dataset so the classifier is very efficient in recognizing it. The other classes instead are often mis-classified. The confusion matrix shows the number of correct predictions for all the classes in the test set. Rows represent the true classes while column the predicted ones. In order to understand better the behaviour of the classifier we evaluate the **classification report** which returns other metrics such as **precision**, **recall** and **f1-score**.

```

1 print("Classification Report:\n")
2 report = classification_report(
3     labels_test,
4     test_predictions,
5     target_names=list(label_names.values()),
6     zero_division=0 # Evita l'errore per classi non
                     previste
7 )
8 print(report)

```

- **Precision:** measures the proportion of correctly predicted positive instances (True Positives) out of all instances predicted as positive (True Positives + False positive).

$$Precision = \frac{Tp}{Tp + Fp} \quad (4.3)$$

- **Recall:** measures the proportion of correctly predicted positive instances (True Positives) out of all actual positive instances (True Positives + False Negative).

$$Recall = \frac{Tp}{Tp + Fn} \quad (4.4)$$

- **F1-score:** is the harmonic mean of precision and recall. It provides a single metric that balances the trade-off between precision and recall, especially useful when the classes are imbalanced.

We can clearly see that classes S and F are totally misclassified by the model, which implies that we are in the case of overfitting. A balanced model should return similar values of precision and recall for each class.

Classification Report:					
	precision	recall	f1-score	support	
N	0.88	0.99	0.93	1796	
S	0.00	0.00	0.00	62	
V	0.75	0.18	0.29	153	
F	0.00	0.00	0.00	21	
Q	0.85	0.71	0.78	157	
accuracy			0.88	2189	
macro avg	0.50	0.38	0.40	2189	
weighted avg	0.84	0.88	0.84	2189	

Figure 4.2: Classification report

The confusion matrix is built upon the information provided by the so-called false positive, false negative, true positive, and true negative for multi-class classification.

- **True Positive:** The number of instances correctly predicted as belonging to the interested class.
- **False Positive:** The number of instances incorrectly predicted as belonging to the interested class, but they actually belong to other classes.
- **False Negative:** The number of instances that actually belong to the interested class but are incorrectly predicted as another class.
- **True Negative:** The number of instances that neither belong to the interested class nor are predicted as that class.

In our case we are mostly interested in true positive and false negative values (FN is more critical as undetected anomalies can lead to life-threatening situations) which are, respectively for each class:

- Class N, True Positive: 1783, False Negatives: 13
- Class S, True Positive: 0, False Negatives: 62
- Class V, True Positive: 27, False Negatives: 126
- Class F, True Positive: 0, False Negatives: 21
- Class Q, True Positive: 112, False Negatives: 45

Chapter 5

Improving the dataset

When examining the class distribution in the dataset using bar plots, it becomes evident that the dataset is significantly imbalanced. This imbalance means that some classes have many more items than others, which can lead to biased model behavior. Ideally, a model should learn from a balanced dataset to avoid favoring the overrepresented classes. To address this, we can modify the dataset to have an equal number of items per class. Here's how this can be achieved.

5.1 Dataset Balancing

The first step ensures that all classes in both training and test datasets have an equal number of samples, mitigating class imbalance issues during training. This process is constrained by the size of the smallest class in the dataset, i.e., 641 items for the training set and 162 items for the test set. While balancing ensures fairness in class representation, it restricts the amount of data available for training, which can affect the model's performance.

5.2 Data Augmentation

To overcome the limitations imposed by the dataset size, we can use data augmentation techniques. These techniques generate synthetic data by modifying existing data samples, thereby:

- Increasing the number of items in the training dataset to provide the model with more diverse examples.
- Enhancing the model's robustness and generalizability, as it learns to handle variations in the input data.

Data augmentation methods are usually applied with some randomness or probability to avoid introducing new biases into the dataset. In order to implement this we will define a class augment:


```

38     # Apply stretch randomly
39     if np.random.binomial(1, 0.5) == 1: #1 means that
        a test has been performed. 0.5 is the
        probability of success of the test.
40                                     # 1 for
                                                success, 0
                                                otherwise
41
42     x = self.stretch(x)
43     performed_augmentations.append('stretch')
44
45     # Apply amplify randomly
46     if np.random.binomial(1, 0.5) == 1:
47         x = self.amplify(x)
48         performed_augmentations.append('amplify')
49
49     return x, performed_augmentations

```

Let's test and visualize the applied augmentations (*stretch* and *amplify*) on a sample signal:

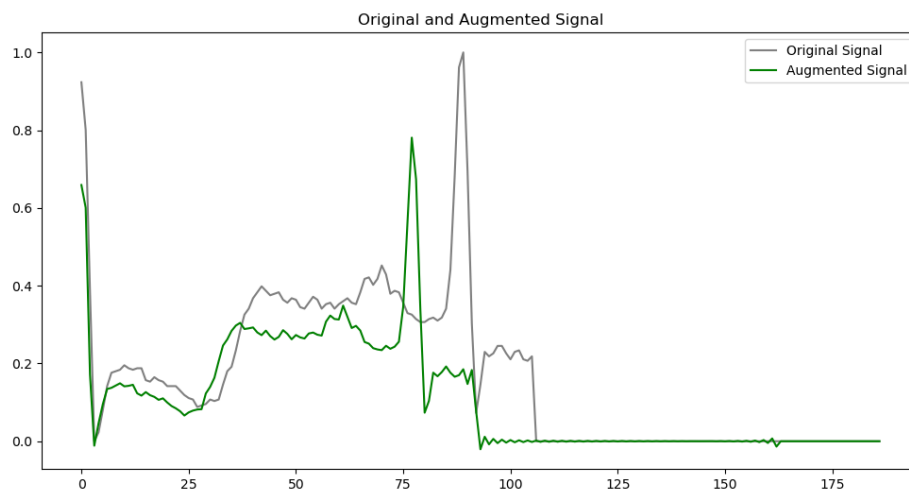


Figure 5.1: Comparison between Original and Augmented Signal

```

1  # Test augment class
2  augmenter = augment()
3
4  # Select a sample signal
5  sample_signal = train_list[0][:-1] # Extract signal
        values without label
6
7  # Apply augmentation
8  augmented_signal, applied_augmentations =
        augmenter.perform(np.array(sample_signal))
9  print("Applied augmentations:", applied_augmentations)
10

```

```

11
12 # Plot
13 # Visualize the original and augmented signals
14 plt.figure(figsize=(12, 6))
15 plt.plot(sample_signal, label="Original Signal",
16          color='gray')
17 plt.plot(augmented_signal, label="Augmented Signal",
18          color='green')
19 plt.legend()
20 plt.title("Original and Augmented Signal")
21 plt.show()

```

To apply such augmentations to our train dataset:

```

1 n_aug = 100 # number of waveforms per class that we want
2             # to add to our dataset
3
4 augmenter = augment()
5
6 for cls in df.iloc[:, -1].unique(): # 5 classes
7     # extract n_aug samples for each class
8     class_samples = df.loc[df.iloc[:, -1] ==
9                             cls].sample(n=n_aug, random_state=16)
10
11     #Applies augmentation for each sample in the class
12     for idx, row in class_samples.iterrows(): #500 samples
13         original_signal = row.iloc[:-1].values # Remove
14         label and trasform into list
15         augmented_signal, _ =
16             augmenter.perform(original_signal)
17
18         #Adding the augmented signal and the class
19         new_row = list(augmented_signal) + [cls] #
20             concatenate the signal with the label
21         new_row_df = pd.DataFrame([new_row],
22                                     columns=train_df.columns) # otherwise I cannot
23             use concatenate
24         train_df = pd.concat([train_df, new_row_df])

```

In the end we will obtain this class distribution in train_df:

- N : 641
- S : 641
- V : 641
- F : 641
- Q : 641

Normalizing the Dataset

`train_list` and `test_list` are scaled to a range of $(-1, 1)$ using the `MinMaxScaler`. The scaling is fitted on the training set and applied to both sets. Here's a comparison between the original and normalized version of a signal.

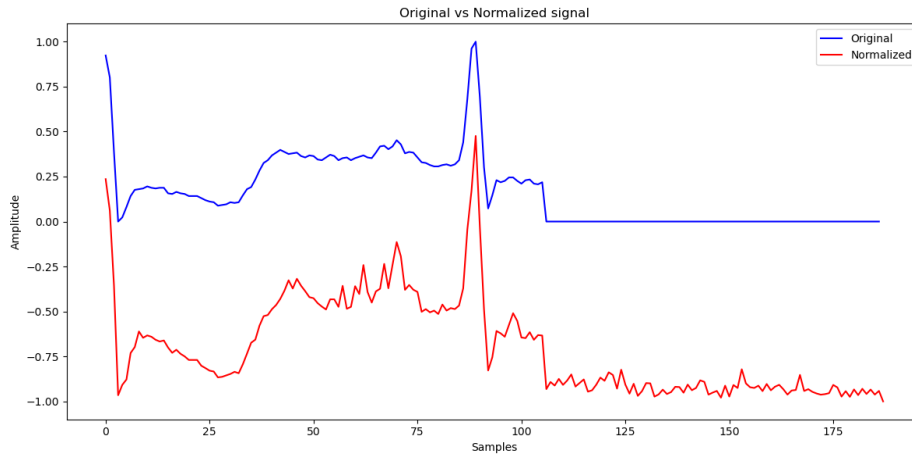


Figure 5.2: Comparison between Original and Normalized Signal

```
1 scaler = MinMaxScaler(feature_range=(-1, 1))
2 train_set = scaler.fit_transform(train_list)
3 test_set = scaler.transform(test_list)
```

Feature Vector Compilation

Combines all extracted features (e.g., ZCR, spectral flux) into a fixed-length vector. This vector is computed for every signal in `train_set` and `test_set`.

```
1 train_fvector.append(compute_feature_vector(x, Fs, N, H))
```

5.3 Additional possible Augmentations

To further enrich the dataset and enhance the model's ability to generalize, several additional augmentation techniques can be applied. These methods aim to introduce realistic variations to the data, mimicking real-world conditions or artifacts, and improving robustness against diverse input scenarios. Here are some potential augmentations:

- **Noise Injection:** Add low-level Gaussian noise to the signal to simulate realistic measurement conditions, such as electrical interference or sensor imperfections. This technique helps the model become less sensitive to noise and improves its ability to generalize.

- **Time Shifting:** Shift the signal slightly forward or backward in time, mimicking variations in acquisition timing or misalignments during measurement. This can make the model more robust to slight temporal discrepancies in the input data.
- **Filtering:** Apply low-pass or high-pass filters to the signal to emulate conditions where certain frequency components are attenuated, such as baseline wandering or power-line interference. This augmentation ensures the model can handle signals with varying frequency characteristics
- **Frequency Warping:** Slightly modify the signal's frequency components to simulate variability in heart rate or signal acquisition. This technique can introduce subtle changes in the waveform morphology, helping the model adapt to physiological variations

After applying augmentations and normalizing the dataset, the resulting data should be well-prepared for training the classification model. The incorporation of augmentation techniques not only increases the size and diversity of the dataset but also enhances the models resilience to variations and noise. Additionally, balancing the classes ensures that the classification metrics, such as precision, recall, and F1-score, are more representative of the model's performance across all categories. This comprehensive preparation significantly improves the model's reliability and effectiveness in real-world applications.

Chapter 6

Tuning of hyperparameters and final results

It's time to train our new classifier on the augmented dataset and perform the classification in order to verify if the new dataset is actually able to provide us with a better classification.

6.1 Accuracy

The primary goal is to assess the impact of dataset augmentation on the classification performance by comparing the accuracy of the model trained on the new dataset with the results obtained from the original dataset. Additionally, we aim to evaluate whether the augmented dataset reduces overfitting. After modifying the dataset, the accuracy dropped to 70%, which is 19% less than the previous accuracy of 89%. The lower accuracy suggests that the augmented dataset has introduced more variability, which likely made the model less confident on the test set. The accuracy decrease aligns with expectations, as augmented datasets generally aim to make the model more robust rather than achieving peak performance on specific test data. With the new dataset, we expect the model to generalize better to unseen data, thus addressing overfitting. The augmented dataset successfully mitigated overfitting but at the expense of test accuracy. While the 70% accuracy is lower than the 89% previously achieved, it reflects a model that is less reliant on specific patterns in the training data.

6.2 Confusion matrix and classification report

To analyze the performance of the classifier by examining the confusion matrix and computing key classification metrics, including precision, recall, and F1-score. Additionally, we compare the results before and after dataset augmentation to evaluate improvements or regressions. The classifier performs more consistently across all classes after dataset augmentation. While the overall accuracy has dropped from 89% to 71%, recall and F1-scores are more equitable,

reflecting better generalization. This improvement in balance indicates that the augmented dataset helps mitigate overfitting.

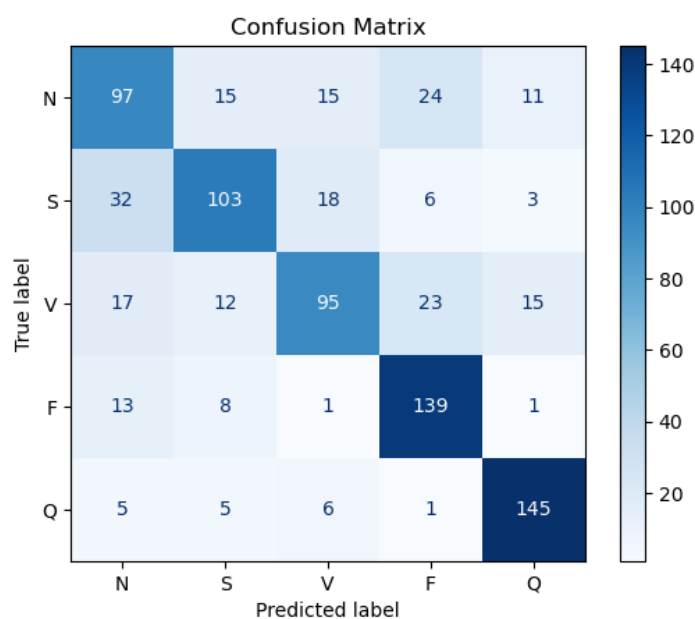


Figure 6.1: Confusion Matrix

Classification Report:				
	precision	recall	f1-score	support
N	0.59	0.60	0.60	162
S	0.72	0.64	0.68	162
V	0.70	0.59	0.64	162
F	0.72	0.86	0.78	162
Q	0.83	0.90	0.86	162
accuracy			0.71	810
macro avg	0.71	0.71	0.71	810
weighted avg	0.71	0.71	0.71	810

Figure 6.2: Classification report

We also evaluate True Positive and False Negative values for each class:

- **N**: True Positive: 97, False Negatives: 65
- **S**: True Positive: 103, False Negatives: 59
- **V**: True Positive: 95, False Negatives: 67
- **F**: True Positive: 139, False Negatives: 23
- **Q**: True Positive: 145, False Negatives: 17

6.3 Tuning Hyperparameters

Tuning C

C is a regularization parameter in SVM that controls the trade-off between maximizing the margin and minimizing the classification error:

- **Small C:** Places higher importance on margin maximization, leading to a simpler model that may underfit.
- **Large C:** Prioritizes classification accuracy on the training data, potentially causing overfitting.

To evaluate the impact of the hyperparameter C on the SVM classifier's performance while keeping the kernel fixed we analyze performance metrics for $C = (0.1, 1, 100, 1000, 10000, 100000)$.

```

1      # Test different C values and analyze performance
2  C_values = [0.1, 1, 100, 1000, 10000, 100000]
3  for C in C_values:
4      svc = SVC(C=C, kernel='rbf')
5      svc.fit(train_fvector, labels_train)
6      test_predictions = svc.predict(test_fvector)
7      accuracy = accuracy_score(labels_test,
8                                test_predictions)
9      print(f"Accuracy for C={C}: {accuracy*100:.2f}%")
10     # You can also print confusion matrices for each C
11     ConfusionMatrixDisplay.from_estimator(
12         svc,
13         test_fvector,
14         labels_test,
15         display_labels=list(label_names.values()),
16         cmap="Blues"
17     )
18     plt.title("Confusion Matrix")
19     plt.show()
20     #Classification report
21     report = classification_report(labels_test,
22                                   test_predictions,
23                                   target_names=list(label_names.values()))
24     print("Classification Report:\n", report)

```

The following accuracies were obtained for different C values (more metrics in the code):

- Test accuracy for $C = 0.1$: 60.49%
- Test accuracy for $C = 1$: 64.32%
- Test accuracy for $C = 100$: 74.20%

- Test accuracy for $C = 1000$: 78.77%
- Test accuracy for $C = 10000$: 81.48%
- Test accuracy for $C = 100000$: 79.01%

As C increases, accuracy initially improves, indicating better model performance. The highest accuracy is achieved at $C=10000$ (81.48%). In a Support Vector Machine (SVM), the parameter C , as we already seen, controls the trade-off between maximizing the margin (making the decision boundary as wide as possible) and minimizing classification errors on the training set (penalizing misclassified points). A higher C puts more weight on correctly classifying training data points. The SVM model prioritizes reducing misclassification errors over maximizing the margin. As a result, it focuses more on fitting the training data. The model becomes less tolerant of outliers or noise. It tries harder to correctly classify even points that are close to or within the margin, potentially improving training accuracy. Increasing C can also reduce bias in the model because it forces the decision boundary to align more closely with the training data. However, increasing C does not always lead to better accuracy. It can cause the model to overfit the training data, particularly if the dataset contains noise or outliers. Overfitting reduces generalization performance, which may decrease accuracy on unseen test data. In fact for $C=100000$ accuracy decreases slightly.

Best model performance

After extensive hyperparameter(Kernel and relative parameters, C , N , H) optimization, the best performing model was configured as follows:

- Kernel: Radial Basis Function (RBF)
- C : 10,000
- gamma: 0.1
- N : 128
- H : 32
- Accuracy: 81.98

```

1 # Dictionary for memorizing results
2 results = {}
3 # Hyperparameter tuning
4 for kernel in ['rbf', 'linear', 'poly', 'sigmoid']:
5     for C in [0.1, 1, 10, 100, 1000, 10000]:
6         for n in [128]: # the highest value of N
7             for h in [32, 64]: # H should be be N/2 or N/4
8
9                 # feature vector

```



```

10     train_fvector = []
11     test_fvector = []
12
13     for x in tqdm(train_set): # Feature
14         vector for training set
15         f_vector = compute_feature_vector(x,
16             Fs, n, h)
17         train_fvector.append(f_vector)
18
19     for x in tqdm(test_set): # Feature vector
20         for test set
21         f_vector = compute_feature_vector(x,
22             Fs, n, h)
23         test_fvector.append(f_vector)
24
25     # conversion of feature vector in numpy
26     array
27     train_fvector = np.array(train_fvector)
28     test_fvector = np.array(test_fvector)
29
30
31     if kernel == 'linear':
32         # Linear kernel: vary only C
33         g=0
34         d=0
35         svc = SVC(C=C, kernel=kernel)
36         svc.fit(train_fvector, labels_train)
37         test_predictions =
38             svc.predict(test_fvector)
39         accuracy = accuracy_score(labels_test,
40             test_predictions)
41
42         # saving results in the dictionary
43         params = (kernel, C, n, h, g, d)
44         results[params] = accuracy
45         print(f"Accuracy for kernel={kernel}
46             C={C} N={n} H={h} gamma={g}
47             degree={d}: {accuracy*100:.2f}%")
48
49     elif kernel == 'poly':
50         # Polynomial kernel: vary C and degree
51         g=0
52         for d in [2, 3, 4]:
53             print(d)
54             svc = SVC(C=C, kernel=kernel,
55                 degree=d)
56             svc.fit(train_fvector,
57                 labels_train)

```

```

47         test_predictions =
48             svc.predict(test_fvector)
49         accuracy =
50             accuracy_score(labels_test,
51                             test_predictions)
52
53         params = (kernel, C, n, h, g, d)
54         results[params] = accuracy
55         print(f"Accuracy for
56               kernel={kernel} C={C} N={n}
57               H={h} gamma={g} degree={d}:
58               {accuracy*100:.2f}%")
59
60     elif kernel == 'rbf':
61         # RBF kernel: vary C and gamma
62         d=0
63         gammas= [0.01, 0.1,1]
64         for g in gammas:
65             print(g)
66             svc = SVC(C=C, kernel=kernel,
67                       gamma=g)
68             svc.fit(train_fvector,
69                     labels_train)
70             test_predictions =
71                 svc.predict(test_fvector)
72             accuracy =
73                 accuracy_score(labels_test,
74                                 test_predictions)
75
76             params = (kernel, C, n, h, g, d)
77             results[params] = accuracy
78             print(f"Accuracy for
79                   kernel={kernel} C={C} N={n}
80                   H={h} gamma={g} degree={d}:
81                   {accuracy*100:.2f}%")
82
83     elif kernel == 'sigmoid':
84         # Sigmoid kernel: vary C and gamma
85         d=0
86         for g in [0.01, 0.1, 1]:
87             svc = SVC(C=C, kernel=kernel,
88                       gamma=g)
89             svc.fit(train_fvector,
90                     labels_train)
91             test_predictions =

```

```

79         svc.predict(test_fvector)
80         accuracy =
81             accuracy_score(labels_test,
82                             test_predictions)
83
84         params = (kernel, C, n, h, g, d)
85         results[params] = accuracy
86         print(f"Accuracy for
87             kernel={kernel} C={C} N={n}
88             H={h} gamma={g} degree={d}:
89             {accuracy*100:.2f}%")
90
91 best_params = max(results, key=results.get)
92 best_accuracy = results[best_params]
93
94 print("\nBest parameters and accuracy:")
95 print(f"Kernel: {best_params[0]}, C: {best_params[1]}, N:
96     {best_params[2]}, H: {best_params[3]}")
97 print(f"Accuracy: {best_accuracy*100:.2f}%")

```

The performance on the test set achieved an accuracy of 81.98%, which suggests the model effectively generalized to unseen data.

The metrics observed show a smaller accuracy value in respect to the initial one. The Dataset balancing and the augmentation techniques helped the model becoming more consistent and robust on each class.

The RBF kernel confirmed itself as the best kernel. It considers the distance between data points, focusing on how close they are to one another. This localization means it adapts better to varying densities or clusters in the data. We can see that some classes are better recognized from the model, especially class F and Q, while the other three classes have a smaller f1-score. The model has still some biases towards those better recognized classes.

6.4 Improvement possibilities

To improve the classification and, in particular, the selected metrics, several strategies can be adopted, focusing on both the dataset and the model.

- **Exploring New Features and Advanced Feature Extraction:** features are critical for model performance. Poorly chosen or insufficiently informative features can limit even the best algorithms. Employing automated feature extraction methods like Principal Component Analysis (PCA) or autoencoders to uncover latent factors in the data could improve the training set. Applying techniques like Fourier Transforms or

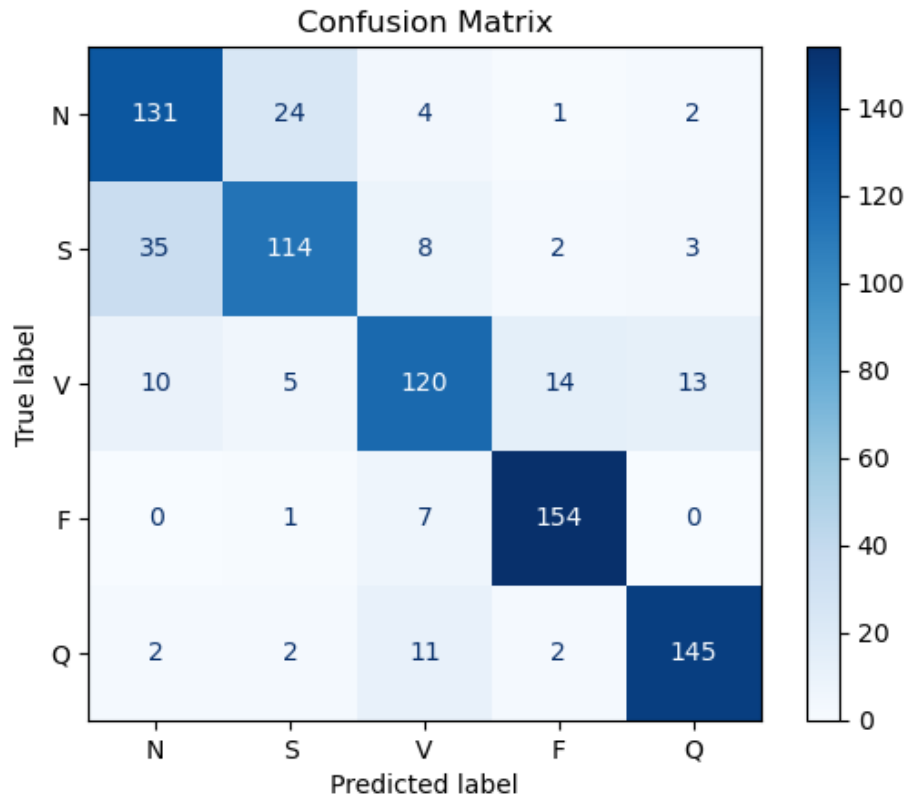


Figure 6.3: Confusion matrix of the best model

Wavelet Transforms to capture more detailed, high-dimensional representations could also be an option.

- **Larger, Balanced, and Realistic Datasets** An imbalanced or insufficient dataset introduces bias and reduces generalizability. Collecting a larger dataset with enough examples for each class to ensure statistical robustness would surely improve the results.
- **Leveraging Advanced Machine Learning Models:** More sophisticated models can capture complex relationships in the data that simpler models might miss. The use of *Deep Learning* models (e.g., Convolutional Neural Networks for image data or Recurrent Neural Networks for sequential data) if the dataset is large enough to justify their use could help recognizing patterns that simpler models can't handle. Also implementing transfer learning with pre-trained networks to benefit from features learned on related tasks could help maximizing the accuracy and stability of the task.
- **More Comprehensive Hyperparameter Tuning:** Poorly chosen hyperparameters can limit a model's performance significantly, indeed using techniques like grid search or random search to systematically explore combinations of hyperparameters are often more usefull. Also ap-

Classification Report:					
	precision	recall	f1-score	support	
N	0.74	0.81	0.77	162	
S	0.78	0.70	0.74	162	
V	0.80	0.74	0.77	162	
F	0.89	0.95	0.92	162	
Q	0.89	0.90	0.89	162	
accuracy			0.82	810	
macro avg	0.82	0.82	0.82	810	
weighted avg	0.82	0.82	0.82	810	

Figure 6.4: Classification report of the best model

plying Bayesian optimization or genetic algorithms for smarter and more efficient hyperparameter exploration and regularizing models using appropriate techniques (e.g., L1/L2 regularization) to prevent overfitting could improve the performance of the model.

- **Cross-Validation:** Using techniques like k-fold cross-validation to assess the model's performance more reliably and reduce the risk of overfitting to the test set.
- **Error Analysis:** Analyze model errors to identify systematic patterns of misclassification. This can guide further feature engineering or model adjustments.